Ders 4 Genişletilmiş Notu: Öğrenci Sunumları: Mikroservislerde Test, Üretime Hazır Servisler ve AWS Lambda ile Dağıtım

Ders 4 Genişletilmiş Notu: Öğrenci Sunumları: Mikroservislerde Test, Üretime Hazır Servisler ve AWS Lambda ile Dağıtım

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin akışı
- 2. Mikroservis mimarisinde test stratejileri
- 3. Servisler için birim testleri
- 4. Servisler için entegrasyon testleri
- 5. Bileşen testleri
- 6. Uçtan uca testler
- 7. Güvenli servis geliştirme
- 8. Yapılandırılabilir servis tasarımı
- 9. Gözlemlenebilir servis tasarımı
- 10. Mikroservis şasisi ve service mesh
- 11. AWS Lambda ve API Gateway ile RESTful servis dağıtımı

1. Dersin akışı

Bu ders baştan sona öğrenci sunumlarından oluşur. Hoca, haftanın özellikle test konusuna ayrıldığını, testle ilgili üç ünitenin işleneceğini ve önceki hafta sunum yapamayan bir öğrencinin sunumunun da eklenmesiyle toplam on sunum yapılacağını söyleyerek başladı. Sunuculardan her sunumu on dakika civarında tutmalarını istedi ve konuların birbirine bağlı olduğunu hatırlattı: bir önceki sunumda anlatılan bir şey varsa üzerinde fazla durmadan "bunu arkadaşımız anlatmıştı" deyip geçmek zaman kazandırır.

Sunumlar sırasıyla şunlardır: mikroservis mimarileri için test stratejileri (Bölüm 2), servisler için birim testleri (Bölüm 3), entegrasyon testleri (Bölüm 4), bileşen testleri (Bölüm 5), uçtan uca testler (Bölüm 6), güvenli servisler (Bölüm 7), yapılandırılabilir servisler (Bölüm 8), gözlemlenebilir servisler (Bölüm 9), mikroservis şasisi (Bölüm 10) ve AWS Lambda ile RESTful servis dağıtımı (Bölüm 11). Sunan kişiler bu notta "bir öğrenci" diye anılır. Hoca bu derste sunumları yönetti, ayrıca akademik bir konu anlatmadı.

Güvenlik, yapılandırma, gözlemlenebilirlik ve şasi sunumları İngilizce yapıldı ve bu kısımların altyazısı büyük ölçüde bozuktur (tekrar eden anlamsız satırlar). Bu bölümler ağırlıkla ekrandaki slaytlardan kurulmuştur; sözlü açıklaması okunamayan yerler işaretlenmiştir.

> **Ek bilgi (derste anlatılmadı):** Sunumlardaki şekiller, kod listeleri ve FTGO / Order Service örnekleri Chris Richardson'ın *Microservices Patterns* kitabından alınmıştır. Bu kitapta test konuları büyük olasılıkla dokuzuncu ve onuncu bölümlerde ("Testing microservices", birinci ve ikinci kısım), güvenlik, yapılandırma, gözlemlenebilirlik ve şasi on birinci bölümde ("Developing production-ready services"), dağıtım ise on

ikinci bölümde yer alır. Bileşen testi sunumunun slayt başlığındaki “10.2” numarası da bu bölümlemeye uyar.

2. Mikroservis mimarisinde test stratejileri

İlk sunum İngilizce yapıldı ve test konusunun genel çerçevesini kurdu. Sunucu, yetersiz test edildiği için yıkılabilecek bir bina görseliyle başladı: mimaride olduğu gibi yazılım mimarisinde de test önemli bir aşamadır ve bu aşamanın ihmal edilmesi, iş bittikten sonra önlenemeyen büyük sorunlara ve kayıplara yol açabilir.

2.1 Test nedir, neyi test eder?

Bir testin amacı, test edilen sistemin (SUT, System Under Test) davranışını doğrulamaktır. SUT tek bir sınıf kadar küçük de olabilir, bütün uygulama kadar büyük de.

2.2 Otomatik testin dört aşaması

Ekrandaki slayta göre otomatik bir test genellikle dört aşamadan oluşur (ekrandan, 00:03:40):

- **Setup:** Test fixture’ını istenen başlangıç durumuna getirmek.
- **Exercise:** SUT’u çağırmak.
- **Verify:** Çağrının sonucu ve SUT’un durumu hakkında iddialarda (assertion) bulunmak.
- **Teardown:** Gerekirse test fixture’ını temizlemek.

Slayttaki şekil bu yapıyı şöyle gösterir: bir test çalıştırıcı (test runner) bir test sınıfını yürütür. Test sınıfında birden çok test metodu vardır. Her test metodu Setup, Execute, Verify ve Teardown adımlarından geçer ve bu adımlar, içinde SUT’un bulunduğu fixture üzerinde işler.

```
Test runner --executes--> Test class
                          ├─ Test method
                          ├─ Test method --> Setup    --> Fixture [SUT]
                          │                  Execute  --> Fixture [SUT]
                          │                  Verify
                          │                  Teardown --> Fixture
                          └─ Test method
```

2.3 Test türleri ve test piramidi

Sunum, bir uygulamanın ya da servisin işlevsel yönlerini doğrulayan otomatik testlere odaklanır ve dört test türünü sayar: birim testi, entegrasyon testi, bileşen testi ve uçtan uca test (ekrandan, 00:06:00).

Test piramidinin temel fikri, piramitte yukarı çıkıldıkça daha az test yazmaktır: çok sayıda birim testi, az sayıda uçtan uca test geliştirilmelidir. Piramit alttan üste şöyle dizilir (ekrandan, 00:08:00):

Katman	Görevi
Unit	İş mantığını test etmek
Integration	Servisin bağımlılıklarıyla iletişim kurduğunu doğrulamak
Component	Bir servisin kabul testleri
End-to-end	Bir uygulamanın kabul testleri

Piramidin yanındaki ok, alt katmanların hızlı, güvenilir ve ucuz; üst katmanların yavaş, kırılgan ve pahalı olduğunu gösterir.

2.4 Servis etkileşim türleri ve tüketici güdümlü sözleşme testi

Sunucu, Order Service'in diğer servislerle etkileşim türlerini sıraladı: Order Service ve Order History Service arasında domain event ile publish/subscribe (Order Service yayıncı, Order History Service tüketici) ve Order Service'in çeşitli servislere komut mesajı gönderip yanıtları tükettiği asenkron istek/yanıt.

Tüketici sözleşme testi (consumer contract test), bir sağlayıcının API'sinin tüketicinin beklentilerini karşıladığını kontrol eden bir entegrasyon testidir. Tüketici ile sağlayıcı arasındaki etkileşim, sözleşme (contract) denen bir dizi örnekle tanımlanır (ekrandan, 00:10:00). Slayttaki şekilde API Gateway takımı, Order History Service takımı ve başka bir servisin takımı, her biri Order Service için kendi sözleşme test takımını (contract test suite) yazar. Bu test takımları Order Service'in dağıtım hattının (deployment pipeline) içinde çalışır ve Order Service'i test eder. Sunucuya göre bu test takımı, API'nin kullanıcının ihtiyaçlarını karşıladığını garanti eder; diğer takımların sunduğu testler de sağlayıcının dağıtım hattında çalıştırılır. Bu testler için Spring Cloud Contract kullanılabilir.

Spring Cloud Contract ile yazılmış bir sözleşme örneği, bir HTTP isteği ve bir HTTP yanıtından oluşur (ekrandan, 00:12:00): istekte metot `GET` ve yol `/orders/1223232`; yanıtta durum kodu `200`, `Content-Type: application/json;charset=UTF-8` başlığı ve bir JSON gövdesi bulunur. Aynı örnek entegrasyon testi sunumunda tam gövdesiyle tekrar gösterildi (Bölüm 4.4).

3. Servisler için birim testleri

İkinci sunum, servisler için birim testi yazmayı anlattı. Hedefler şunlardı: birim testini tanımlamak ve yazmak, birim testlerin uçtan uca ve entegrasyon testlerinden farkını anlamak, solitary ve sociable testleri ve bunlarla ilişkili test double kavramını anlamak, servisler için birim testlerinin nasıl yazıldığını görmek.

3.1 "Birim" nedir?

Sunucu, birim kavramını ölçü birimlerine benzetti: metrik sistemde ölçek büyükse kilometreden, küçükse metreden ya da santimetreden söz ederiz. Birim testinde de "birim" nasıl tanımladığımıza bağlıdır, ama genelde kodun çok küçük bir parçasıdır. Nesneye yönelik tasarım yapılıyorsa birim genelde bir sınıftır; fonksiyonel programlamada ise bir fonksiyon ya da metot birim olarak alınır.

Sunucuya göre birim testlerinin asıl amacı hata avlamak değildir; hata da bulurlar, ama temelde beklenen sonuç ile elde edilen sonucun karşılaştırılmasıdır. Küçük bir metodun beklendiği gibi çalışıp çalışmadığı sınanır. Çok küçük bir parça test edildiği için birim testleri çok hızlı, milisaniyeler içinde çalışmalıdır. Testler elle yazılır ama genelde bir çerçeveden yararlanılır: önyüzde Jest, Java'da JUnit, Python'da pytest ve unittest. Birim testleri test piramidinin en altındadır: en küçük parçaları, en hızlı ve en izole biçimde test ederler.

3.2 Solitary ve sociable birim testleri

Birim testleri birimi diğer parçalardan ayrı test eder. Ama test edilen kod doğal olarak başka sınıfların metotlarını, hatta başka sınıfların tamamını kullanır. Bu durumda iki seçenek vardır:

- **Sociable (sosyal) birim testi:** Bağımlılıklar gerçek halleriyle çalıştırılır.
- **Solitary (yalnız) birim testi:** Bağımlılıkların yerine mock ya da stub denen test double'lar kullanılır.

Slayttaki şekilde soldaki gerçek bileşen birçok bağımlılıkla iç içedir. Test sırasında bu bağımlılıklar "taklit edilir" (fake): sanki çalışıyormuş gibi davranırlar, ama ürettikleri sonucun içi test için önemli değildir; önemli olan, test edilen metodun onları doğru çağırmasıdır. Çerçeveler mock'un içini yapılandırmaya izin verir: o metodun ne döndürmesi gerektiğini bildiğimiz için mock'u o değeri döndürecek şekilde ayarlarız.

Sunucu mock ile stub arasında bir fark olduğunu, bazı kaynaklarda ikisinin birbirinin yerine kullanıldığını söyledi. Ona göre mock bir nesnenin tamamını taklit eder, stub ise yalnız bir nesnenin davranışıyla, genelde tek tek metotlarla ilgilidir.

> **Ders sonrası not:** Sunucu mock ile stub farkını "mock nesnenin tamamını, stub tek bir davranışı taklit eder" diye verdi. Yaygın kullanımda (Martin Fowler'ın "Mocks Aren't Stubs" yazısı ve Gerard Meszaros'un sınıflaması) ayrım büyük olasılıkla başka bir eksende yapılır: stub test sırasında yapılan çağrılara önceden hazırlanmış cevaplar döndürür ve durum doğrulamasına hizmet eder; mock ise hangi çağrıların beklendiğini taşır ve davranış (etkileşim) doğrulamasına hizmet eder.

3.3 İlk örnek: iki stringi birleştiren birim

Sunucu basit bir örnekle başladı (ekrandan, 00:20:00):

```
public class MyUnit {

    public String concatenate(String one, String two){
        return one + two;
    }
}

public class MyUnitTest {

    @Test
    public void testConcatenate() {
        MyUnit myUnit = new MyUnit();

        String result = myUnit.concatenate("one", "two");

        assertEquals("onetwo", result);
    }
}
```

Java'da testler için genelde ayrı bir test sınıfı oluşturulur ve sınıf adında "Test" kelimesi bulunur. Her test metodunun başına `@Test` notasyonu yazılır; yazılmazsa metot test olarak çalışmaz. Test metodunun adı test edilen metodun adına göre verilir: `concatenate` için `testConcatenate`. Böylece test kendi kendini belgeler. `MyUnit` içinde başka metotlar varsa, `MyUnitTest` içinde her biri için yine `@Test` notasyonlu yeni metotlar yazılır. Bu örnekte nesne her testin içinde oluşturulur; sunucu bunun ileride görülecek `@Before` notasyonlu bir metoda da taşınabileceğini söyledi.

Test, `concatenate("one", "two")` çağrısının sonucunun `"onetwo"` olduğunu `assertEquals(beklenen, gerçek)` ile doğrular. Sunucu başarısız bir durumu da gösterdi: metot bir şekilde yalnız `one` döndürürse (örneğin `two` eklenmeyi unutulursa ya da sonradan silinirse) test başarısız olur ve çerçeve beklenen değer ile gerçek değeri yan yana gösterir. Bu çıktı, kodda hatanın nerede olduğuna geri dönüp bakmayı kolaylaştırır.

Başka assertion metotları da vardır: `assertTrue`, `assertFalse`, `assertNotEquals` gibi. Yalnız doğru çalışmayı değil, bir şeyin olmaması gerektiğini de test etmek mümkündür. Jest'te `expect` ile `toBe` gibi eşleyiciler birlikte kullanılır; `toHaveBeenCalled` metotların çağrıldığını kontrol eder; `expect(...).not` ise bir şeyin olmadığını doğrular.

3.4 Servislerde hangi test türü?

Bir servisteki sınıfların rolü ve sorumluluğu, hangi testin (sociable ya da solitary) yazılacağını belirler. Dışarıya çok API isteği yapan sınıflar için solitary testler tercih edilir; bu istekler test sırasında taklit edilir. Daha basit sınıflar, örneğin entity'ler, sociable testlerle sınanabilir.

3.5 Entity testi: Order

Sunucunun ilk servis örneği `Order` entity'sinin `getOrderTotal` metodunu test eden `OrderTest` sınıfıdır. Önceki örnekte nesne testin içinde oluşturulmuştu; ama test sayısı arttıkça aynı kurulumun her testte tekrar yazılması gerekir. Bunun yerine `@Before` notasyonlu bir `setUp` metodu yazılır; bu metot her testten önce çalışır ve ortak nesneleri hazırlar. Test metodunun adı (`shouldCalculateTotal` benzeri) yine ne test ettiğini anlatmalıdır. Test, `getOrderTotal`'ın nasıl

çalışması gerektiğini bildiğimiz için beklenen toplamı bir çarpma işlemiyle elle hesaplar ve metodun sonucuyla eşit olduğunu doğrular. Sunucu bu testi, basit sınıflar için önerdiği sociable test örneği olarak verdi.

3.6 Controller testi: OrderController

Controller için solitary bir test örneği verildi (ekrandan, 00:25:15):

```
public class OrderControllerTest {
    private OrderService orderService;
    private OrderRepository orderRepository;

    @Before
    public void setUp() throws Exception {
        orderService = mock(OrderService.class);
        orderRepository = mock(OrderRepository.class);
        orderController = new OrderController(orderService, orderRepository);
    }

    @Test
    public void shouldFindOrder() {
        when(orderRepository.findById(1L))
            .thenReturn(Optional.of(CHICKEN_VINDALOO_ORDER_));
        given().
            standaloneSetup(configureControllers(
                new OrderController(orderService, orderRepository))).
        when().
            get("/orders/1").
        then().
            statusCode(200).
            body("orderId",
                equalTo(new Long(OrderDetailsMother.ORDER_ID).intValue()));
    }
}
```

Test edilen sınıf `OrderController`'dır; `OrderService` ve `OrderRepository` ise mock'lanır, çünkü bunlar gerçek çağrılar ve istekler yapar. Asıl test edilen controller mock'lanmaz, gerçek nesne olarak oluşturulur.

`shouldFindOrder` testinin ilk satırı mock'un nasıl yapılandırıldığını gösterir: `orderRepository.findById(1L)` çağrıldığında depo, kimliği 1 olan siparişi (`CHICKEN_VINDALOO_ORDER_`) döndürecektir. Ardından REST Assured çerçevesiyle controller'a bir `GET /orders/1` isteği yapılır. Bu istek gerçekten bir sunucuya gitmez; sunucunun açıkladığı gibi veritabanını bozmak istemeyiz. Bu örnekte istek `GET`'tir, ama özellikle `POST` istekleri gerçek veriyi değiştirebilir. Son olarak yanıtın durum kodunun 200 olduğu ve gövdedeki `orderId` alanının beklenen sipariş kimliğine eşit olduğu doğrulanır.

3.7 Birim testlerinin değerlendirmesi

Sunucu birim testlerini şöyle özetledi. Küçük bir parçayı, diğer sınıflardan izole biçimde test ederler. Bunun avantajları vardır: hızlı çalışırlar, hızlı geri bildirim verirler, daha kolay yazılırlar ve hataları erken bulmaya yardım ederler. Dezavantajları da vardır: bir noktadan sonra hızlı ve kaliteli birim testi yazmak zorlaşır ve birim testleri bütün problemleri kavramaz. Entegrasyon testi olmadıkları için birimler arasındaki ilişkileri görmezler; aradaki bir ilişki bozulursa bunu gözden kaçırabilirler.

Bazı kaynaklar birim testlerini entegrasyon testlerinden önce çalıştırmayı, entegrasyon testlerinde ise birimler arasındaki ilişkileri birleştirerek sınamayı önerir. Birim testleri otomatikleştirildiği için çoğu yerde test güdümlü geliştirmeye (TDD) de yol açar: testler koddan önce yazılır. Sunucuya göre bu, mimarinin bir haritası ve belgesi gibi işler; her birimin adı ve o birimden ne beklendiği önceden yazılmış olur. Bu da iyi bir yazılım mimarisi geliştirme yöntemi olabilir.

4. Servisler için entegrasyon testleri

Üçüncü sunum entegrasyon testlerini anlattı: entegrasyon testinin ne olduğu, stratejileri ve sözleşme türleri.

4.1 Tanım ve piramitteki yeri

Entegrasyon testleri, bir servisin veritabanları ve diğer uygulamalar gibi altyapı servisleriyle doğru biçimde etkileşime girebildiğini doğrulayan testlerdir. Birim testleri bu etkileşimleri sınamaz; entegrasyon testleri bu boşluğu doldurur.

Servisler genellikle başka servislerle etkileşir. Slayttaki Order Service şeması bunu gösterir (ekrandan, 00:30:00): API Gateway (tüketici) içindeki `Order Service proxy`, Order Service'in `Order controller`'ına (sağlayıcı) bağlanır. Order Service'in `Order aggregate event publisher`'ı bir olay kanalı (event channel) üzerinden Order History Service'in `Order history event handlers` sınıfına olay gönderir. Order Service içindeki `Kitchen Service proxy` (tüketici) komut kanalı (command channel) üzerinden Kitchen Service'in `Kitchen Service command handler`'ına (sağlayıcı) komut gönderir ve yanıt kanalından (reply channel) cevap alır. `Order Repository` ise veritabanına bağlanır. Şekilde test edilen sınıfların her birine ayrı bir test bağlanmıştır. Entegrasyon testleri bu etkileşimlerin doğru çalıştığını görmek için yazılır.

Entegrasyon testleri test piramidinde birim testlerinin bir üstündeki katmandır. Uçtan uca testlerin aksine bütün servisleri başlatmazlar; bunun yerine testleri önemli ölçüde basitleştiren ve etkinliklerini azaltmayan stratejiler kullanırlar.

4.2 İki strateji

- **Adaptörleri tek tek test etmek:** Servisin her adaptörü, destekleyici sınıflarıyla birlikte test edilir. Böylece servisin tamamı yerine az sayıda sınıf birlikte sınanır ve testler çok daha basit olur.
- **Sözleşme kullanmak:** Sözleşme, bir servis çifti arasındaki etkileşimin somut bir örneğidir.

4.3 Etkileşim türleri ve sözleşmeler

Etkileşim türü	Tüketici	Sağlayıcı	Sözleşme
REST tabanlı istek/yanıt	API Gateway	Order Service	HTTP istek ve yanıtı
Publish/subscribe	Order History Service	Order Service	Domain event
Asenkron istek/yanıt	Order Service	Kitchen Service	Komut mesajı ve yanıt mesajı

Sözleşme hem tüketiciyi hem de sağlayıcıyı test etmek için kullanılır ve iki tarafın API üzerinde anlaşmasını sağlar. Kullanımı, hangi tarafın test edildiğine göre biraz değişir:

- **Tüketici tarafı testleri:** Tüketicinin adaptörüne yönelik testlerdir. Sağlayıcıyı simüle eden stub'ları yapılandırmak için sözleşmeleri kullanırlar. Böylece çalışan bir sağlayıcı gerektirmeyen tüketici entegrasyon testleri yazılabilir.
- **Sağlayıcı tarafı testleri:** Sağlayıcının adaptörüne yönelik testlerdir. Adaptörün bağımlılıkları için mock kullanırlar ve adaptörü sözleşmelerle sınarlar.

4.4 Kalıcılık entegrasyon testleri

Daha önce yazılan birim testleri yalnız bellekteki nesneleri sınar. Bir servisin doğru çalıştığından emin olmak için veritabanına erişim mantığının beklendiği gibi çalıştığını doğrulayan kalıcılık (persistence) entegrasyon testleri de yazılmalıdır. Aşamaları şunlardır:

- **Kurulum (setup):** Veritabanı şeması oluşturulup bilinen bir duruma getirilerek veritabanı kurulur; ayrıca bir veritabanı işlemi (transaction) başlatılabilir.

- **Yürütme (execute):** Bir veritabanı işlemi gerçekleştirilir.
- **Doğrulama (verify):** Veritabanının durumu ve veritabanından okunan nesneler hakkında iddialarda bulunulur.
- **Teardown:** Kurulumda başlatılan işlemi geri alarak veritabanında yapılan değişiklikleri geri alan isteğe bağlı bir aşamadır. Sunucuya göre bu aşama yapılmasa da olur, ama ilk üç aşama zorunludur.

4.5 REST tabanlı istek/yanıt etkileşimleri

REST, servisler arası iletişimde yaygın bir mekanizmadır. REST istemcisi ile REST servisi, REST uç noktalarını ve istek ile yanıt gövdelerinin yapısını içeren bir REST API üzerinde anlaşmalıdır. İstemci doğru uç noktaya doğru HTTP isteğini göndermeli, servis de beklenen yanıtı geri göndermelidir.

Slayttaki Spring Cloud Contract şemasında iki taraf vardır: tüketici (consumer) tarafı ve sağlayıcı (provider) tarafı. Tüketici tarafı bir istek/yanıt mesajı gönderir, sağlayıcı kendi içinde yazılmış sınıfıyla doğru cevabı döner. Sözleşme örneği şudur (ekrandan, 00:35:15):

```
org.springframework.cloud.contract.spec.Contract.make {
    request {
        method 'GET'
        url '/orders/1223232'
    }
    response {
        status 200
        headers {
            header('Content-Type': 'application/json;charset=UTF-8')
        }
        body('''{"orderId" : "1223232", "state" : "APPROVAL_PENDING"}''')
    }
}
```

Bu sözleşme şunu söyler: `/orders/1223232` yoluna bir `GET` isteği gelirse sağlayıcı 200 durum koduyla, JSON içerik türüyle ve siparişin kimliğini ve `APPROVAL_PENDING` durumunu içeren bir gövdeyle cevap vermelidir.

4.6 Publish/subscribe etkileşimleri

Servisler çoğu zaman bir ya da daha fazla başka servisin tükettiği domain event'ler yayınlar. Bu yapıda mesajı yayınlayan bir yayıncı (publisher) ve mesajları dinleyen bir abone (subscriber) vardır. Örneğin Order Service bir `Order` aggregate'i oluşturulduğunda ya da güncellendiğinde sipariş olayları yayınlar; Order History Service bu olayların tüketicilerinden biridir. Bu iki servisin doğru etkileşebildiğini doğrulayan testler yazılmalıdır.

Spring Cloud Contract ile yazılan örnekte bir tarafta olayı tetikleyen (trigger) bir yayıncı, karşı tarafta bu yayını dinleyen bir sınıf vardır. Sözleşmedeki mesaj, REST örneğindeki gibi JSON türünde bir gövde ve başlıklarla tanımlanır.

4.7 Asenkron istek/yanıt etkileşimleri

Asenkron istek/yanıt etkileşiminde iki taraf vardır: komutu gönderen istek sahibi (requestor) ve komutu işleyip bir yanıt geri gönderen yanıtlayıcı (replier). İki taraf komut mesaj kanalının adı ile komut ve yanıt mesajlarının yapısı üzerinde anlaşmalıdır. Spring Cloud Contract ile kurulan şemada sağlayıcı tarafında bir komut kanalı (command channel) ve bir yanıt kanalı (reply channel) üzerinde anlaşılır ve testler için bu kanallar tanımlanır.

4.8 Özet

Entegrasyon testleri servisler arasındaki etkileşimleri test eder. Uçtan uca testler gibi bütün sistemi çalıştırmazlar; yalnız servisler arası etkileşimlerin doğru olup olmadığını sınarlar. Bu yüzden uçtan uca testlerden daha hızlı çalışırlar.

5. Bileşen testleri

Dördüncü sunum bileşen (component) testi geliştirmeyi anlattı.

5.1 Bileşen testi ve kabul testi

Ekrandaki slayta göre bir servis için kabul testleri yazmanın çok daha iyi bir yolu bileşen testi kullanmaktır. Bileşen testleri test piramidinde entegrasyon testleri ile uçtan uca testler arasındadır. Bileşen testi bir servisin davranışını yalıtılmış olarak doğrular; sonuç olarak bileşen testlerini yazmak çok daha kolay, çalıştırmak daha hızlıdır (ekrandan, 00:40:00). Şekilde bir bileşen testi bir servisi test eder; servisin her bağımlılığı bir stub ile değiştirilmiştir.

Kabul testleri, bir yazılım bileşeni için işe dönük testlerdir. Kullanıcı hikâyelerinden ya da kullanım durumlarından üretilirler. Örneğin Order Service için önemli hikâyelerden biri sipariş verme hikâyesidir: "Order Service'in bir tüketicisi olarak sipariş verebilmeliyim." Bu hikâye bir senaryoya genişletilebilir:

```
Given  geçerli bir tüketici
Given  geçerli bir kredi kartı kullanılarak
Given  restoran siparişleri kabul ediyor
When   Ajanta'da Chicken Vindaloo için sipariş veriyorum
Then   sipariş onaylanmalı
And    bir OrderAuthorized olayı yayınlanmalı
```

Her senaryo bir kabul testi tanımlar. Bu senaryo için iki çağrı yapılır: önce `POST /orders` uç noktası çağrılarak bir sipariş oluşturulur, sonra `GET /orders/{orderId}` uç noktası çağrılarak siparişin durumu doğrulanır.

5.2 Gherkin ve Cucumber

Gherkin, çalıştırılabilir spesifikasyonlar yazmak için kullanılan bir DSL'dir (alana özgü dil). Gherkin ile kabul testleri yukarıdaki gibi senaryolarla tanımlanır; ardından Gherkin için bir test otomasyon çerçevesi olan Cucumber ile bu spesifikasyonlar çalıştırılır.

Order Service gibi bir servis için Gherkin spesifikasyonu bir dizi özellikten (feature) oluşur; her özellik bir dizi senaryoyla tanımlanır. Order Service için sipariş verme, siparişi iptal etme ve siparişi düzeltme gibi özellikler vardır. Sipariş verme özelliğinde iki senaryo örneği verildi:

- **Siparişin onaylanması:** Geçerli tüketici, geçerli kredi kartı ve siparişleri kabul eden restoran; sipariş onaylanır ve bir `OrderAuthorized` olayı yayınlanır.
- **Siparişin reddedilmesi:** Geçerli tüketici, süresi dolmuş bir kredi kartı ve siparişleri kabul eden restoran; kartın süresi dolduğu için sipariş reddedilir ve bir `OrderRejected` olayı yayınlanır.

Cucumber, Gherkin ile yazılmış testleri çalıştıran otomatik bir test çerçevesidir ve Java dahil birçok dilde vardır. Cucumber for Java kullanılırken bir adım tanımları (step definitions) sınıfı yazılır. Bu sınıfın metotları `@Given`, `@When`, `@Then` gibi notasyonlarla işaretlenir; her metot, senaryodaki bir adımın anlamını tanımlar (ekrandan, 00:43:25):

```
public class StepDefinitions ... {
  ...
  @Given("A valid consumer")
  public void useConsumer() { ... }
  @Given("using a(.?) (.*) credit card")
  public void useCreditCard(String ignore, String creditCard) { ... }
  @When("I place an order for Chicken Vindaloo at Ajanta")
  public void placeOrder() { ... }
  @Then("the order should be (.*)")
  public void theOrderShouldBe(String desiredOrderState) { ... }
  @And("an (.*) event should be published")
  public void verifyEventPublished(String expectedEventClass) { ... }
}
```

Notasyonlardaki düzenli ifade grupları senaryo metninden değer yakalar ve bunları metot parametrelerine aktarır. Örneğin "using a valid credit card" adımında `creditCard` parametresi `valid` olur; "the order should be approved" adımında `desiredOrderState` parametresi `approved` olur.

5.3 Bileşen testlerinin tasarımı

Bir bileşen testinin Gherkin senaryosunu çalıştırabilmesi için önce servisi çalıştırması ve servisin bağımlılıklarını ayarlaması gerekir. Order Service yalıtılmış biçimde test edileceği için bileşen testi diğer servisler için stub'lar yapılandırmalı, ayrıca bir veritabanı ve mesajlaşma altyapısı kurmalıdır. Burada gerçekçilik ile hız ve basitlik arasında takas yapan birkaç seçenek vardır:

- **Süreç içi (in-process) bileşen testi:** Servisin bağımlılıkları için bellek içi stub'larla çalışır. Örneğin Order Service için bir test H2, HSQLDB ya da Derby gibi bellek içi bir veritabanı ve bellek içi stub'lar kullanacak şekilde yapılandırılır. Avantajı yazmasının daha basit ve çalışmasının daha hızlı olmasıdır; dezavantajı dağıtılabilir servisin kendisini test etmemesidir.
- **Süreç dışı (out-of-process) bileşen testi:** Veritabanları ve mesaj aracıları gibi gerçek altyapı servislerini kullanır, ama uygulama servisi olan bütün bağımlılıklar için stub kullanır. Önemli yararı test kapsamını geliştirmesidir, çünkü test edilen şey dağıtılana çok daha yakındır. Dezavantajı yazmasının daha karmaşık, çalışmasının daha yavaş olması ve süreç içi bileşen testinden potansiyel olarak daha kırılgan olmasıdır.

5.4 Order Service için bileşen testleri

Order Service'in bileşen testleri, Gherkin kabul testi DSL'iyle yazılmış senaryoları Cucumber ile çalıştırır. Testler, Order Service'i Apache Kafka ve bir veritabanı gibi altyapı servisleriyle birlikte çalıştırmak için Docker kullanır.

Kredi kartı adımının tanımı şöyle gösterildi; slaytın başlığına göre `@Given useCreditCard()` metodu "verilen kredi kartı kullanılarak" adımının anlamını tanımlar (ekrandan, 00:46:15):

```
@ContextConfiguration(classes =
    OrderServiceComponentTestStepDefinitions.TestConfiguration.class)
public class OrderServiceComponentTestStepDefinitions {
  ...
  @Autowired
  protected SagaParticipantStubManager sagaParticipantStubManager;
  @Given("using a(.?) (.*) credit card")
  public void useCreditCard(String ignore, String creditCard) {
    if (creditCard.equals("valid"))
      sagaParticipantStubManager
          .forChannel("accountingService")
          .when(AuthorizeCommand.class).replyWithSuccess();
    else if (creditCard.equals("invalid"))
      sagaParticipantStubManager
          .forChannel("accountingService")
          .when(AuthorizeCommand.class).replyWithFailure();
    else
      fail("Don't know what to do with this credit card");
  }
}
```

Sunucu bu kodu şöyle açıkladı: gelen kredi kartı bilgisi kontrol edilir. Kart geçerliyse accounting servisinin stub'ı `AuthorizeCommand` komutuna başarılı yanıt verecek şekilde ayarlanır. Geçersizse başarısız yanıt verecek şekilde ayarlanır. İkisi de değilse test, kartla ne yapılacağının bilinmediğini söyleyen bir hata mesajıyla başarısız olur. Burada accounting servisi gerçek bir servis değil, saga katılımcısı stub'ıdır; bileşen testinin "bağımlılıklar stub" yaklaşımı tam olarak budur.

5.5 Bileşen testlerini çalıştırma

Testler Order Service'i ve bağımlılıklarını çalıştırmak için Docker kullanır. Docker konteyneri, bir servis örneğini yalıtılmış bir sanal alanda dağıtmayı sağlayan hafif bir işletim sistemi sanallaştırma mekanizmasıdır. Docker Compose ise bir dizi konteyner tanımlayıp onları bir birim olarak başlatıp durdurmayı sağlayan çok kullanışlı bir araçtır. Örnek uygulamanın kök dizininde bütün servisler ve altyapı servisleri için konteynerleri tanımlayan bir Docker Compose dosyası vardır.

Gradle yapılandırmasında iki şey yapılır. Birincisi, Gradle Docker Compose eklentisi bileşen testlerinden önce çalışacak ve Order Service'i, yapılandırdığı altyapı servisleriyle birlikte başlatacak şekilde ayarlanır. İkincisi, Docker imajının gerektirdiği JAR dosyasının önce oluşturulması için ilgili görev, bileşen testi görevine bağımlı hale getirilir. Bileşen testleri tek bir komutla çalıştırılır. Bu komut sırasıyla servisi derler, servisi ve altyapı servislerini başlatır, testleri çalıştırır ve çalışan servisleri durdurur.

6. Uçtan uca testler

Beşinci sunum uçtan uca (end-to-end) testleri anlattı.

6.1 Tanım

Önceki sunumlarda görüldüğü gibi sistemin içindeki servisler önce kendi içlerinde test edilir; bunların ardından sistemde uçtan uca test uygulanır. Uçtan uca test bir uygulamanın akışını baştan sona test etmek için kullanılır ve sistemin bütününün kontrolünü sağlar: test edilen sistemdeki bütün bileşenlerin birbiriyle uyumlu çalıştığını denetler.

Ekrandaki slayta göre bileşen testi her servisi ayrı test eder, uçtan uca test ise bütün uygulamayı test eder. Uçtan uca testler test piramidinin tepesindedir, çünkü yavaş, kırılgan ve geliştirmesi zaman alıcı testlerdir. Çok sayıda hareketli parçaları vardır: birden çok servis ve onların altyapı servisleri dağıtılmalıdır. Bu yüzden yavaştırlar. Çok sayıda servisin dağıtılması gerektiği için birinin başlamama olasılığı da yüksektir; bu da testleri güvenilmez kılar. Sonuç olarak uçtan uca testlerin sayısı en aza indirilmelidir (ekrandan, 00:50:00).

6.2 Kullanıcı yolculuğu testleri

Uçtan uca testler kullanıcı yolculuğu (user journey) testleri olarak yazılır. Kullanıcı yolculuğu, kullanıcının sisteme girişinden (login) itibaren bütün yolculuğunu simüle eden bir yapıdır: kullanıcı uygulamaya girer, ardından yapılabilecek işlemleri yapar. Bu akışı kontrol eden test, kullanıcı yolculuğu testidir.

Bu testler işe dönüktür. Doğrudan müşteri tarafından görülmezler, ama işin etkin yönetimi için önemlidirler ve o işi yapan grubun anlayabileceği bir dilde yazılmalıdırlar. Bu nedenle bunlar da Gherkin diliyle yazılır; notasyonu önceki sunumda görülmüştü.

Örnek senaryo şöyledir: müşteri bir sipariş oluşturabilmeli, siparişi revize edebilmeli ve sonunda iptal edebilmelidir.

```
Given  geçerli bir müşteri, geçerli kredi kartı bilgisi
       ve siparişleri kabul eden geçerli bir restoran
When   bir ürün sipariş edilir
Then   sipariş onaylanmalı ve fiyat belirlenmeli
When   sipariş iki ürün daha eklenerek revize edilir
Then   fiyatın güncellendiği görülmeli
When   sipariş iptal edilir
Then   siparişin iptal onayı görülmeli
```

6.3 Çalıştırma ve maliyet

Uçtan uca testler de Gradle Docker Compose eklentisiyle çalıştırılır: eklenti gerekli bütün servislerin aynı anda çalışmasını sağlar ve uçtan uca testleri mümkün kılar. Sunucuya göre örnek sipariş uygulamasının uçtan uca testleri yaklaşık 4-5 dakika sürmüştür. Daha büyük ve

daha çok testi olan uygulamalarda bu süre çok daha uzar; yani uçtan uca testler çok ağır ve yavaş testlerdir. Bu yüzden yazılan kullanıcı yolculuğu testleri hem kapsayıcı hem de az sayıda olmalıdır ki bütün sistem kolay ve hızlı biçimde test edilebilsin. Özetle: kullanıcı yolculuğu testleri en aza indirilmeli, bütün sistemin uyumluluğu az sayıda testle sağlanmalıdır.

Hoca da bu sunumun kısa olmasının doğal olduğunu, çünkü Gradle ve Docker Compose ile çalıştırma kısmının önceki bileşen testi sunumunda zaten anlatıldığını belirtti.

7. Güvenli servis geliştirme

Altıncı sunum İngilizce yapıldı ve üretime hazır servislerin ilk yönü olan güvenliği anlattı. Hoca bu sunumla on birinci üniteye geçildiğini söyledi.

7.1 Giriş

Sunucu, siber güvenliğin bütün işletmeler için kritik bir sorun haline geldiğini söyledi. Güvenli yazılım geliştirmek için donanım güvenliği, verinin iletimde ve yerel bilgisayarlarda şifrelenmesi, kimlik doğrulama (authentication) ve yetkilendirme (authorization) ile yazılım bakım politikaları gibi ciddi konuların ele alınması gerekir.

Ekrandaki slayta göre mikroservis tabanlı mimari karmaşık ve gelişen sistemlerde giderek yaygınlaşmaktadır. Mikroservisler bileşenleri değiştirmeyi ya da yükseltmeyi çok kolaylaştırır ve birden çok geliştiricinin bütün çözümün farklı yönleri üzerinde birbirini etkilemeden çalışmasına izin verir. Ancak mikroservis mimarisinin kendine özgü güvenlik zorlukları vardır; onu kullanmak isteyen biri, verinin gizli ve güvenli kalması ve sistemin gerektiğinde ayakta olması için bu zorlukları baştan dikkatle incelemelidir (ekrandan, 00:55:30).

7.2 Monolitik güvenlik neden aktarılamaz?

Bir mikroservis uygulamasının güvenlik tasarımı, monolitik bir uygulamanınkinin kopyası olamaz. Monolitik güvenlik mimarisinin iki parçası mikroservislerle uyumsuzdur:

- **Bellek içi güvenlik bağlamı (in-memory security context):** Servisler belleği paylaşmaz, bu yüzden bellek içi bir güvenlik bağlamı kullanamazlar. Mikroservis mimarisinde kullanıcı kimliğini bir servisten diğerine taşımak için farklı bir mekanizma gerekir.
- **Merkezi oturum (centralized session):** Teoride birden çok servis veritabanında tutulan bir oturuma erişebilir; ama bu, gevşek bağlılık (loose coupling) ilkesini çiğner.

7.3 API Gateway'de kimlik doğrulama

Ekrandaki slayta göre API Gateway, kullanıcılardan gelen istekleri servislere iletmeden önce doğrular. Bunun sonucunda bir güvenlik açığı olasılığı çok daha düşüktür. Bir başka yararı, farklı kimlik doğrulama mekanizmalarıyla yalnız API Gateway'in uğraşmasıdır; bu karmaşıklık servislerden gizlenir (ekrandan, 00:58:30).

7.4 Yetkilendirme

Bir uygulama, istemcinin istenen işlemi yapmaya yetkili olduğunu doğrulayan bir yetkilendirme mekanizması uygulamalıdır. Bir kullanıcının belirli bir servise erişim izni yoksa API Gateway isteği servise iletmeden reddedebilir. Kimlik doğrulamada olduğu gibi, yetkilendirmeyi API Gateway'de merkezileştirmek güvenlik açığı riskini azaltır (ekrandan, 01:00:00).

7.5 Kullanıcı kimliğini ve rolleri JWT ile taşımak

Kullanıcı kimliğini servisler arasında taşımak için iki tür token vardır (ekrandan, 01:02:00):

- **Opaque token:** Dezavantajları performansı ve erişilebilirliği düşürmesi ve gecikmeyi artırmasıdır.
- **JSON Web Token (JWT):** Kullanıcı kimliği ve roller gibi iddiaları (claim) iki taraf arasında güvenle temsil etmenin standart bir yoludur. Dezavantajı, kötü niyetli bir üçüncü tarafın eline geçerse geri alınamamasıdır.

Slayta göre bu sorun OAuth 2.0 adlı bir güvenlik standardıyla çözülür. Sunucu, OAuth 2.0 kullanmanın önemli bir yararının bunun kanıtlanmış bir güvenlik standardı olması olduğunu söyledi: hazır bir OAuth 2.0 yetkilendirme sunucusu kullanmak, güvenli olmayan bir tasarım geliştirmeye zaman ve risk harcamamak demektir. Ancak OAuth 2.0 mikroservis mimarisinde güvenliği uygulamanın tek yolu değildir.

> **Ek bilgi (derste anlatılmadı):** Opaque token yalnız bir referanstır; servis token'ın kime ait olduğunu öğrenmek için her seferinde onu veren sunucuya sormak zorundadır, bu da sunucudaki ek çağrı yüzünden gecikme ve tek hata noktası demektir. JWT ise iddiaları imzalı biçimde kendi içinde taşır, bu yüzden servis onu yerinde doğrulayabilir; ama süresi dolana kadar geçerli kalır. OAuth 2.0'da bu sorun büyük olasılıkla kısa ömürlü erişim token'ları ile daha uzun ömürlü, geri alınabilir yenileme (refresh) token'larının birlikte kullanılmasıyla hafifletilir.

7.6 Özet

Sunucu şu sonuçla bitirdi: bir servis işlevsel gereksinimlerini karşılarken aynı zamanda güvenli, yapılandırılabilir ve gözlemlenebilir olmalıdır. Mikroservis mimarisindeki güvenliğin birçok yönü monolitik mimariden farklı değildir. Farklı olan, kullanıcı kimliğinin API Gateway ile servisler arasında nasıl iletildiği ve kimlik doğrulama ile yetkilendirmeden kimin sorumlu olduğudur.

8. Yapılandırılabilir servis tasarımı

Yedinci sunum İngilizce yapıldı ve yapılandırılabilir servis tasarımını anlattı: giriş, dışsallaştırılmış yapılandırmanın iki modeli ve Spring Cloud Config.

8.1 Neden dışsallaştırılmış yapılandırma?

Servis geliştirme, çeşitli işlevsel gereksinimleri karşılamak için en önemli süreçlerden biridir. Bir servisi çalıştırmak için çeşitli yapılandırma özellikleri gerekir ve bu özelliklerin değeri servisin çalıştığı ortama bağlıdır. Servisler genellikle mesaj aracısı (message broker) ve veritabanı gibi bir ya da daha fazla dış servis kullanır. Bu yüzden servise yapılandırma özelliklerini çalışma anında sağlayan dışsallaştırılmış yapılandırma (externalized configuration) deseni kullanılmalıdır.

8.2 Push tabanlı model

Push modeli, dağıtım ortamı ile servisin iş birliğine dayanır: dağıtım ortamı, bir servis örneği oluştururken yapılandırma özelliklerini sağlar. Slayttaki örnekte dağıtım altyapısı bir Order History Service örneği oluştururken ortam değişkenlerini ayarlar; servis bu ortam değişkenlerini okur (ekrandan, 01:08:30):

```
BOOTSTRAP_SERVERS=kafka1:9092
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=...
AWS_REGION=...
```

Alternatif olarak hazırlık (staging) ortamında bir yapılandırma dosyası da kullanılabilir; servis örneği yapılandırma özelliklerini başlangıçta bu dosyadan okur.

8.3 Pull tabanlı model

Pull modelinde servis örneği yapılandırma özelliklerini bir yapılandırma sunucusundan okur. Başlangıçta servis örneği yapılandırma servisine kendi yapılandırmasını sorar; örnekte Order History Service `getConfiguration("orderHistoryService")` çağrısıyla Kafka sunucusu ve AWS anahtarları gibi özellikleri alır. Yapılandırma sunucusuna erişmek için gereken özellikler, örneğin sunucunun ağ konumu (`CONFIG_SERVER_URL`), servis örneğine push tabanlı bir mekanizmayla, örneğin ortam değişkenleriyle verilir (ekrandan, 01:10:00).

Yapılandırma sunucusunun yararları şunlardır:

- **Merkezi yapılandırma:** Bütün yapılandırma özellikleri tek bir yerde tutulur, bu da yönetimi kolaylaştırır. Yinelenen özellikleri ortadan kaldırmak için bazı uygulamalar global varsayılanlar tanımlamaya izin verir; bunlar servis bazında ezilebilir.
- **Hassas verinin şeffaf çözülmesi:** Veritabanı parolası gibi hassas verileri şifrelemek iyi bir güvenlik pratiğidir. Bunun bir zorluğu, servis örneğinin bu veriyi genelde çözmesi, yani şifreleme anahtarına sahip olması gerekmesidir. Bazı yapılandırma sunucuları bu çözme işini otomatik yapar.

8.4 Spring Cloud Config

Spring Cloud Config, Spring Cloud projelerinden biridir ve doğrudan merkezi yapılandırma sunucusu desenine karşılık gelir. Mikroservis mimarisinin temel desenlerinden biri, dağıtık bir sistemdeki çeşitli uygulama servislerinin dışsallaştırılmış yapılandırmasının merkezi olarak yönetilmesidir.

Spring Cloud Config'in ilk ana bileşeni, birçok servisin yapılandırmasını yöneten yapılandırma sunucusudur (config server). Sunucu, yapılandırma deposu (config repo) denen bir depolama mekanizmasıyla desteklenir; bu depo bir veritabanı, dosya sistemi ya da Git gibi bir kaynak kod yönetim aracı olabilir. Servisler yapılandırmayı HTTP protokolüyle okur. Spring tabanlı mikroservisler yapılandırma sunucusuyla genelde yapılandırma istemcisi (config client) aracılığıyla etkileşir.

Yapılandırma sunucusunun iskelet kodu Spring Boot Initializr ile alınabilir. Sunucu, yapılandırmanın birden çok depolama modelinde tutulmasını destekleyecek şekilde tasarlanmıştır. Sunucunun kaynağını tanımlamak için yeni bir Git deposu oluşturulur.

9. Gözlemlenebilir servis tasarımı

Sekizinci sunum İngilizce yapıldı ve gözlemlenebilir servislerin desenlerini anlattı.

9.1 Neden gözlemlenebilirlik?

Ekrandaki giriş slaytına göre iki amaç vardır (ekrandan, 01:17:00). Birincisi uygulamanın ne yaptığını bilmektir: saniyedeki istek sayısı ve kaynak kullanımı gibi. İkincisi bir sorun olduğunda uyarı almaktır: başarısız bir servis örneği ya da kullanıcıyı etkilemeden önce hızla dolan bir disk gibi. Uyarıdan sonra sorunun giderilebilmesi ve kök nedenin bulunabilmesi gerekir.

Sunucu bu amaçla altı desen saydı: health check API, log aggregation, distributed tracing, exception tracking, application metrics ve audit logging.

9.2 Health check API

Bir servis örneği, dağıtım altyapısına bir isteği karşılayıp karşılayamayacağını bildirebilmelidir. Bunun için iyi bir çözüm, servisin bir health check uç noktası sunmasıdır.

9.3 Log aggregation

Log aggregation, mikroservis mimarisinde yararlı bir hata ayıklama aracıdır. Slayttaki şekilde Service A'nın iki örneği ve Service B'nin bir örneği, her biri içindeki günlükleme kütüphanesiyle (logging library) günlük üretir. Bu günlükler bir log pipeline'a, oradan bir günlük sunucusuna (logging server) akar. Kullanıcı günlükleri bu sunucuda görüntüler ve sunucu kullanıcıyı uyarabilir (ekrandan, 01:20:00).

```
Service A instance 1 [Logging library] --> Log --\
Service B instance 1 [Logging library] --> Log ----> Log pipeline --> Logging server <--view/r
Service A instance 2 [Logging library] --> Log --/
```

9.4 Distributed tracing ve exception tracking

Distributed tracing: her dış isteğe benzersiz bir kimlik atanır ve isteğin servisler arasındaki akışı izlenir. Sunucuya göre uygulamanın ne yaptığını anlamanın iyi bir yolu budur; performans profili çıkarmaya ve izlemeye yardım eder.

Exception tracking: servisler istisnaları bir istisna izleme servisine raporlar.

9.5 Application metrics

Metrikler altyapı düzeyindeki metriklerden (CPU, bellek ve disk kullanımı) uygulama düzeyindeki metriklere (servis isteği gecikmesi, yürütülen istek sayısı) kadar uzanır. Slayttaki şekilde servis örneğinin içinde uygulama kodu, uygulama çerçevesi ve dil çalışma zamanı vardır; uygulama kodundaki metrik kütüphanesi örnekleri metrik servisinin metrics ingestion bileşenine gönderir. Dağıtım altyapısı da metrik gönderir. Metrik servisi metrikleri bir metrik veritabanında saklar, görselleştirme ve uyarı bileşenleriyle kullanıcıya sunar (ekrandan, 01:23:00).

Bir metrik örneğinin üç temel özelliği vardır: ad, değer ve zaman damgası. Slayttaki örnek:

```
name=cpu_percent
value=68
timestamp=34938934893
dimensions:
   machine=node1
   ...
```

Sunucu adın `cpu_percent`, değerin sayısal `68` ve üçüncü özelliğin örneğin zamanı olduğunu açıkladı. Slayttaki örnekte bunlara ek olarak makine gibi boyutlar (dimensions) da vardır.

9.6 Audit logging

Audit logging, her kullanıcının eylemlerini kaydeder. Her kayıt, kullanıcının kimliğini, yaptığı eylemi ve ilgili iş nesnesini (business object) içerir. Uygulama audit kaydını genelde bir veritabanı tablosunda saklar. Uygulamanın birkaç yolu vardır: audit kaydını iş mantığına eklemek, yönelime dayalı programlama (AOP) kullanmak ve event sourcing kullanmak.

10. Mikroservis şasisi ve service mesh

Dokuzuncu sunum İngilizce yapıldı ve mikroservis şasisi (microservice chassis) desenini anlattı.

10.1 Sorun: tekrarlanan kod

Sunucu önce gözlemlenebilirliğin ana başlıklarını (health check, uygulama metrikleri, dağıtık izleme) hatırlattı. Ardından şasinin çözdüğü sorunu gösterdi: şasi olmadan her servis (Order Service, Consumer Service, diğerleri) kendi içinde aynı katmanları tekrarlar (ekrandan, 01:28:30):

```
Her serviste:
  Service Template Build logic             <- tüm servislerde tekrarlanır
  Service template cross-cutting concerns  <- tüm servislerde tekrarlanır
  Service-specific Build logic
  Service-specific Cross-cutting concerns
  Application logic
```

Şasi deseninde ortak bileşenler bir kez yazılır ve servisler arasında paylaşılır; her servise yalnız kendine özgü bileşenleri kalır. Şasi; günlükleme, ortak mantık, dağıtık izleme, istisna işleme ve güvenlik gibi ortak kesişen kaygıları (cross-cutting concerns) sağlar, servisler bunları şasi çerçevesinden alır.

10.2 Avantajlar

Slayttaki listeye göre şasi deseninin avantajları şunlardır (ekrandan, 01:30:00):

- İş mantığını kesişen kaygılardan ayırmak.
- Yapılandırılabilir bir temel çerçeve kurabilmek.
- Kararlı bir kod tabanı sağlamak.
- Hızlı uygulama geliştirmeyi desteklemek.

- Yeni özellikleri yalnız onlara ihtiyaç duyan servislere bağımsız olarak eklemek.

Sunucu dezavantajlardan da söz etti; bu kısım altyazıda okunamıyor. Java için şasi çerçevesi olarak Spring Boot ve Spring Cloud anıldı.

10.3 Service mesh deseni

Sunumun sonunda işlevlerin mikroservis şasisinden service mesh'e taşınması anlatıldı. Slayttaki şekilde API Gateway, Order Service ve Restaurant Service'in her birinde bir mikroservis şasisi vardır, ama şasi artık daha az işlev taşır: günlükleme, dışsallaştırılmış yapılandırma, uygulama metrikleri, dağıtık izleme, health check. Servislerin altındaki service mesh katmanı ise devre kesici (circuit breaker), servis keşfi (service discovery), güvenli iletişim, akıllı trafik yönlendirme, dağıtık izleme ve yük dengeleme işlevlerini üstlenir. Service mesh'in altında dağıtım altyapısı bulunur (ekrandan, 01:31:30).

> **Ek bilgi (derste anlatılmadı):** Service mesh, servisler arasındaki bütün ağ trafiğini yöneten bir altyapı katmanıdır. Genelde her servis örneğinin yanına yerleştirilen bir vekil (sidecar proxy) ile çalışır; Istio ve Linkerd büyük olasılıkla en bilinen örneklerdir. Bu sayede ağ ile ilgili kaygılar dilden bağımsız hale gelir ve her dil için ayrı şasi kütüphanesi yazma ihtiyacı azalır.

11. AWS Lambda ve API Gateway ile RESTful servis dağıtımı

Son sunum, AWS üzerinde Lambda ve API Gateway kullanarak RESTful bir servisi dağıtmayı uygulamalı olarak gösterdi. Hoca bu konunun aslında dağıtım haftasının konusu olduğunu, ancak tamamen uygulamaya dönük olduğu için bu haftaya alındığını belirtti.

11.1 Örnek mimari

Sunucu önce örnek bir uygulamanın AWS mimarisini gösterdi. Amazon Web Services'te birçok hizmet vardır ve uygulama bunlardan yararlanarak dağıtılabilir. Örnekteki yapıda (altyazıda "Squirrel Bin" diye geçen bir site) web uygulamasının CSS ve JavaScript gibi statik içerikleri S3'te tutulur. İş mantığı Lambda fonksiyonlarındadır. Lambda fonksiyonlarını dışarı açmak için Amazon API Gateway kullanılır. Veritabanı olarak DynamoDB kullanılmıştır.

11.2 Lambda fonksiyonu oluşturma

Önce bir AWS hesabı açılır. Konsolun üstündeki arama kutusuna "Lambda" yazıldığında Lambda fonksiyonlarını oluşturup görüntüleyebileceğimiz ve değiştirebileceğimiz bir sayfa açılır. Bu sayfada "Create function" düğmesine basılır. Açılan ekranda hazır şablonlardan biri seçilebilir ya da sıfırdan başlanabilir; sunucu sıfırdan başladı. Fonksiyon adı yazılır, çalışma zamanı seçilir (sunucu Node.js seçti; Python gibi seçenekler de vardır), mimari seçilir ve "Create function" düğmesine basılır.

Ardından kodun yazılıp geliştirilebileceği bir düzenleyici açılır. Sunucunun örnek kodunda bir kullanıcı listesi vardır; fonksiyon çağrıldığında bu listeyi 200 durum koduyla döndürür.

11.3 API Gateway ile dışarı açma

Fonksiyon yazıldı, ama statik web sitelerinden ya da mobil uygulamalardan çağrılabilmesi için dış dünyaya açılması gerekir. Bunun için Amazon API Gateway kullanılır. Adımlar şöyledir:

- Arama kutusuna "API Gateway" yazılır. Açılan ekranda birden çok seçenek vardır; amaç RESTful bir API olduğu için REST API kutusundaki "Build" düğmesine basılır.
- Açılan ekranda protokol olarak REST seçilir, yeni bir API oluşturulacağı için "New API" seçilir, API'ye bir ad ve açıklama verilir ve "Create API" ile API oluşturulur.
- Önce bir kaynak (resource) oluşturulur: "Actions → Create Resource". Kaynak, Amazon'un vereceği geçici alan adının (ya da kendi alan adımızın) sonunda hangi yol ile çağrılacağını belirler. Sunucu `users` adlı bir kaynak tanımladı.
- Kaynağı Lambda fonksiyonuna bağlamak için "Create Method" ile bir metot (`GET`) eklenir. Entegrasyon türü olarak "Lambda Function" seçilir, fonksiyonun bulunduğu bölge seçilir ve fonksiyon adı (`GetUsers`) yazılır; ad listede çıkar. "Save" ile metot oluşturulur.

- Son olarak “Deploy API” ile API yayına alınır.

Yayındaki `users` uç noktası konsolda görünür. `GET` metoduna tıklandığında uç noktaya dışarıdan nasıl erişileceğini gösteren Invoke URL görülür. Bu adrese istek yapıldığında Lambda fonksiyonunun döndürdüğü kullanıcı listesi gelir.

Dersin Toparlaması

Bu ders, mikroservis mimarisinde testin bütün katmanlarını tek oturumda bir araya getirdi. Test piramidinin en altında, iş mantığını izole ve hızlı biçimde sınayan birim testleri vardır; bağımlılıklar gerçek kullanılırsa test sociable, mock ya da stub ile değiştirilirse solitary olur. Bir üst katmanda entegrasyon testleri, servisin veritabanı ve diğer servislerle etkileşimini sözleşmeler (REST istek/yanıt, domain event, komut/yanıt mesajı) üzerinden sınar. Bileşen testleri bir servisin kabul testlerini, servisin bağımlılıklarını stub ile değiştirerek Gherkin senaryoları ve Cucumber ile çalıştırır. En üstteki uçtan uca testler bütün uygulamayı sınar; yavaş ve kırılgan oldukları için az sayıda, kapsayıcı kullanıcı yolculuğu testleri olarak yazılmalıdır.

Dersin ikinci yarısı üretime hazır servislerin üç niteliğini işledi: güvenlik (kimlik doğrulama ve yetkilendirmenin API Gateway’de merkezileştirilmesi, kimliğin JWT ile taşınması, OAuth 2.0), yapılandırılabilirlik (push ve pull tabanlı dışsallaştırılmış yapılandırma, Spring Cloud Config) ve gözlemlenebilirlik (health check, log aggregation, distributed tracing, exception tracking, metrikler, audit logging). Bu ortak kaygıların her serviste yeniden yazılmaması için mikroservis şasisi, ağ ile ilgili kaygıların altyapıya taşınması için de service mesh tanıtıldı. Son sunum, bir sonraki konu olan dağıtıma uygulamalı bir giriş olarak AWS Lambda ve API Gateway ile bir REST uç noktasının nasıl yayına alındığını gösterdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.